Ingegneria del Software — Soluzione del Tema 13/07/2021

Esercizio 1

Si consideri la seguente classe Java `MusicLibrary` per la gestione di una collezione di brani musicali. Ogni brano è rappresentato dalla classe immutabile `Song` che contiene alcune informazioni tra cui l'artista o gruppo musicale (`Artist`) che esegue il brano e la data (`Date`) in cui il brano è stato registrato, come riportato di seguito.

```
public class MusicLibrary {
  // Ritorna l'insieme di artisti che eseguono almeno un brano nella collezione.
  public /*@ pure @*/ Set<Artist> getArtists();

  // Ritorna la lista di brani eseguiti dall'artista artist.
  // Lancia una UnknownArtistException se la collezione non contiene brani di artist.
  public /*@ pure @*/ List<Song> getByArtist(Artist artist) throws UnknownArtistException;

  // Ritorna l'insieme dei brani eseguiti in data date.
  // Lancia una UnknownDateException se nessun brano della collezione e'
  // stato composto in data date.
  public /*@ pure @*/ Set<Song> getByDate(Date date) throws UnknownDateException;

  // Aggiunge il brano alla collezione.
  public void addSong(Song s);
}

public /*@ pure @*/ class Song {
  // Ritorna l'artista o gruppo che esegue il brano
  public Artist getArtist();

  // Ritorna la data di registrazione del brano
  public Date getDate();
}
```

Domanda a)

Si specifichi in JML il metodo `getByDate()`.

Soluzione

Definiamo `getByDate()` in funzione di `getByArtist()` e `getArtists()`.

```
//@requires date != null
//@
//@ensures \result != null && ! \result.isEmpty() &&
//@ (\forall Artist a; getArtists().contains(a);
//@     (\forall Song s; getByArtist(a).contains(s);
//@        \result.contains(s) <==> date.equals(s.getDate()))) &&
//@ (\forall Song s; \result.contains(s);
//@     s.getDate().equals(date) && getArtists().contains(s.getArtist()) &&
//@     getByArtist(s.getArtist()).contains(s))
//@
//@signals (UnknownDateException e)
//@ (\forall Artist a; getArtists().contains(a);
//@     (\forall Song s; getByArtist(a).contains(s); !date.equals(s.getDate())))
public /*@ pure @*/ Set<Song> getByDate(Date date) throws UnknownDateException;
```

Domanda b)

Si specifichi in JML il metodo `addSong()`.

Soluzione

```
//@requires s != null
//@
//@ensures
//@ getArtists().contains(s.getArtist()) &&
//@ getByArtist(s.getArtist()).contains(s) &&
```

```
//@
//@ (\forall Artist a; \old(getArtists()).contains(a) && !a.equals(s.getArtist());
//@   getArtists().contains(a) && getByArtist(a).size() == \old(getByArtist(a)).size() &&
//@   getByArtist(a).containsAll(\old(getByArtist(a))) &&
//@
//@ !\old(getArtists()).contains(s.getArtist()) ==>
//@   getArtists().size() == \old(getArtists()).size() + 1 &&
//@   getByArtist(s.getArtist()).size() == 1 &&
//@ \old(getArtists()).contains(s.getArtist()) ==>
//@   getArtists().size() == \old(getArtists()).size() &&
//@   getByArtist(s.getArtist()).size() == \old(getByArtist(s.getArtist()).contains(s)) ?
//@                                        \old(getByArtist(s.getArtist()).size()) :
//@                                        \old(getByArtist(s.getArtist()).size()) + 1 &&
//@   getByArtist(s.getArtist()).containsAll(\old(getByArtist(s.getArtist())))
public void addSong(Song s) ;
```

Domanda c)

Si consideri un'implementazione che utilizza un `Set` per contenere le canzoni, come mostrato di seguito.

```
public class MusicLibrary {
  private final Set<Song> songs;
  ...
}
```

Per tale implementazione si definiscano l'invariante di rappresentazione e la funzione di astrazione.

Soluzione

Nell'invariante di rappresentazione escludiamo che il `Set` sia nullo o contenga valori nulli.

```
//@(* Representation invariant RI *)
//@private invariant songs != null &&
//@ (\forall Song s; songs.contains(s); s != null)
```

La funzione di astrazione definisce `getArtists()`, `getByArtist()` in funzione del `Set` usato nella rappresentazione. Gli altri metodi pubblici sono infatti definiti a partire da essi.

```
//@(* Abstraction function AF *)
//@private invariant
//@ (\forall Artist a; ; getArtists().contains(a) <==>
//@    (\exists Song s; songs.contains(s); s.getArtist().equals(a)) ) &&
//@ (\forall Song s; s != null && getArtists().contains(s.getArtist());
//@    getByArtist(s.getArtist()).contains(s) <==> songs.contains(s) )
```

Domanda d)

Si consideri la classe `SortedMusicLibrary` che modifica la specifica del metodo `getByArtist()` per garantire che la lista ritornata contenga le canzoni in ordine di data di registrazione (dalla meno alla più recente). È possibile definire `SortedMusicLibrary` come sottoclasse di `MusicLibrary` in accordo con il principio di sostituzione?

Soluzione

È possibile in quanto la classe si limita a rafforzare la post-considizione del metodo `getByArtist()` della sopraclasse.

Esercizio 2

Si consideri la seguente classe Java:

```
public class RainfallDB {
  private List<Double> data;
  public RainfallDB() {
    data = new ArrayList<Double>();
  }
  public double average() {
    double tot = 0.0;
    for(double rf : data) tot += rf;
    return tot/data.size();
  }
  public RainfallDB addReading(double rainfall) {
    RainfallDB temp = new RainfallDB();
    temp.data.addAll(this.data);
    temp.data.add(rainfall);
    return temp;
  }
}
```

Domanda a)

È possibile invocare i metodi della classe `RainfallDB` da thread paralleli senza rischi di conflitti nell'accesso ai dati? Si motivi la propria risposta e in caso di risposta negativa si spieghi come cambiare il codice della classe per ovviare ai problemi di sincronizzazione identificati.

Soluzione

La classe rappresenta oggetti immutabili e come tale non presenta necessità di ulteriore sincronizzazione.

Domanda b)

Si aggiunga alla classe precedente un metodo:

```
public void clear() {...}
```

che "svuota" la collezione di dati, garantendo che la classe resti correttamente sincronizzata (si indichi se necessario come cambiare gli altri metodi).

Soluzione

Il metodo deve essere implementato come segue (come metodo sincronizzato):

```
public synchronized void clear() {
  data.clear();
}
```

e tutti gli altri metodi andranno anch'essi sincronizzati visto che la classe non rappresenta più oggetti immutabili.

Domanda c)

Si aggiunga un ulteriore metodo:

```
public void clone(RainfallDB db) {
```

che sostituisce agli elementi dell'oggetto corrente (this) quelli contenuti nell'oggetto `db` passato come parametro.

Soluzione

L'implementazione del metodo `clone` va accuratamente sincronizzata per evitare deadlock. Una possibile implementazione è la seguente:

```
public void clone(RainfallDB db) {
  List<Double> temp;
  synchronized(db) {
    temp = new ArrayList<Double>(db.data);
  }
```

```
    synchronized(this) {
      data.clear();
      data.addAll(temp);
    }
  }
```

Esercizio 3

Si consideri un'applicazione per un negozio (reale o virtuale), da realizzare in Java. Una parte dell'applicazione gestisce il calcolo del prezzo finale di vendita del "carrello" dell'acquirente. Il pagamento viene poi gestito da un'altra parte dell'applicazione. A tale scopo, il progetto del sistema prevede una classe `Sale` con due metodi di calcolo (prima dello sconto e dopo lo sconto), oltre ad ulteriori dati e operazioni che qui ignoriamo.

```
public class Sale {
  ...
  public Money totaleNonScontato() { ... }
  public Money totaleScontato() { ... }
  ...
}
```

Il calcolo del totale deve essere effettuato in base al costo dei singoli prodotti moltiplicato per loro quantità, considerando però anche la politica di sconto. Il negozio infatti prevede una serie di iniziative promozionali, non sempre uguali, per cui ad esempio ci potrebbe essere uno sconto, di percentuale variabile, il martedì oppure uno sconto del 5% la sera dopo le 23, oppure un'offerta speciale del 15% il mercoledì per gli anziani, ecc. ecc. Tali politiche non sono interamente previste o prevedibili nella specifica del sistema. Occorre quindi progettare un sistema software che sia in grado di gestire facilmente politiche diverse e che renda possibile introdurre facilmente nuove politiche di sconto, o variazioni delle politiche esistenti.

Si utilizzi un design pattern opportuno per la progettazione del sistema. Si tratteggino in Java le classi necessarie e le loro relazioni, utilizzando opportuni frammenti di codice o diagrammi UML per definire, illustrare ed esemplificare il sistema così progettato e il suo funzionamento.

Soluzione

Una politica di sconto è naturalmente un algoritmo. Occorre quindi progettare un sistema che permetta di scegliere a runtime fra diversi algoritmi e che consenta l'aggiunta di nuovi algoritmi. Si tratta quindi senza dubbio di applicare il pattern Strategy. A tale scopo introduciamo un'interfaccia:

```
public interface IPoliticaSconto {
  public Money getTotal(Sale)
}
```

Ogni politica di sconto è una classe che implementa l'interfaccia. Esempio:

```
public class ScontoDelMartedi implements IPoliticaSconto {
  private percentuale;

  public ScontoDelMartedi(int sconto) {
     float percentuale = sconto / 100.0;
  }

  public Money getTotal(Sale s) {
    import java.time.LocalDate;
    LocalDate localDate = LocalDate.now();
    if (localDate.getDayOfWeek()==Tuesday) {
      return new Money(s.totaleNonScontato().value() * (1-percentuale));
    } else {
      return s.totaleNonScontato();
    }
  }
}

public class Sale {
  private IpoliticaSconto politica;
  ...
  public void inizializzaPoliticaSconto(IPoliticaSconto politica) {
    this.politica = politica;
  }

  public Money totaleScontato() {
    return politica.getTotal(this);
  }
  ...
}
```

Esercizio 4

Si considerino i seguenti metodi Java e se ne completino le parti omesse, *utilizzando escusivamente i concetti e i costrutti della programmazione funzionale*.

Domanda a)

Il metodo seguente prende come argomento una lista di persone (`Person`) e restituisce una lista delle persone con più di 60 anni.

Si ipotizzi che la classe `Person` abbia il metodo `getAge()` che restituisce l'età della persona.

```
public static  List<Person> older(List<Person> people) {
  List<Person> elder =  ... ;
  return elder;
}
```

Soluzione

```
public static  List<Person> older(List<Person> people) {
  List<Person> elder =
  people.stream()
    .filter(person -> person.getAge() > 60)
    .collect(Collectors.toList());
  return elder;
}
```

Domanda b)

Il metodo seguente prende come argomento una lista di nomi e stampa il nome più lungo, se la lista non è vuota, altrimenti non stampa nulla.

```
public static  void longest(List<String> names) {
  final Optional<String> theLongest = ... ;
  theLongest.ifPresent(......);
}
```

Soluzione

```
public static  void longest(List<String> names) {
  final Optional<String> theLongest = friends.stream()
     .reduce((name1, name2) ->
        name1.length() >= name2.length() ? name1 : name2);
  theLongest.ifPresent(name ->
        System.out.println(String.format("The longest name: %s", name)));
}
```